Zero-Trust High School Management System: Conceptual Design, Relational Normalization, and Advanced SQL Implementation

K. Siva Aditya (cb.sc.u4aie24126), M. Jayesh (cb.sc.u4aie24128), N. Srinivas (cb.sc.u4aie24140), M.V. Mukesh (cb.sc.u4aie24138)

Department of Artificial Intelligence Engineering

Amrita Vishwa Vidyapeetham, Coimbatore, India

Course: 23AIE303 – Database Management Systems (Team 17 | Batch 2)

Abstract—Academic management in secondary schools involves intricate, highly interdependent data workflows spanning students, faculty allocations, parent-guardian linkages, curriculum courses, attendance tracking, assignments, continuous assessment marks, and compliance logs. In legacy educational software, un-normalized schemas frequently manifest severe update, insertion, and deletion anomalies, while coarse access controls expose sensitive student transcripts to unauthorized tampering. This project presents the comprehensive database design and implementation of the Zero-Trust High School Management System built upon PostgreSQL. The database models twelve core entities adhering strictly to First Normal Form (1NF), Second Normal Form (2NF), Third Normal Form (3NF), and Boyce-Codd Normal Form (BCNF). Zero-Trust security principles are implemented at the database layer through role-based access control, cryptographic password hashing via scrypt, and an immutable, append-only audit logging mechanism capturing every data mutation with timestamp and client IP origin. The relational engine is queried using 59 parameterized SQL endpoints executing multi-table JOINs, analytical aggregations, and atomic ACID transaction blocks. Experimental verification demonstrates complete data integrity, zero SQL injection vulnerability, and full referential consistency across all academic operations.

Index Terms—Relational Database Management Systems, Entity-Relationship Modeling, Normalization, Boyce-Codd Normal Form, Advanced SQL, Role-Based Access Control, Zero-Trust Architecture, Audit Logging.

I. INTRODUCTION

Relational Database Management Systems (RDBMS) constitute the foundational data infrastructure for institutional administration, enterprise resource planning, and academic record systems. In high school environments, educational operations require the coordinated management of multiple stakeholder hierarchies: administrators governing school policies, instructors conducting courses and assigning marks, learners engaging with curricular assessments, and parents monitoring student academic progress.

Traditional school record-keeping systems predominantly suffer from architectural decoupling and un-normalized relational schemas. When school records are maintained across disconnected spreadsheets or flat desktop databases, redundancy accumulates rapidly, leading to synchronization failure, inconsistent grading records, and loss of historical academic continuity. Furthermore, the absence of fine-grained database privileges exposes confidential student records to unauthorized modification.

To resolve these structural vulnerabilities, this project presents the Zero-Trust High School Management System. Developed using PostgreSQL 16 as the underlying relational engine, the system centralizes all academic entities within a strictly normalized relational schema. Operating under the security mandate 'Never Trust, Always Verify', the database enforces role-based authorization, domain constraints, cascading referential integrity, and comprehensive transaction auditing.

II. PROBLEM STATEMENT AND OBJECTIVES

A. Problem Statement

Educational institutions encounter severe administrative deficiencies when operating on un-normalized, legacy database architectures:

1) Data Redundancy and Modification Anomalies: Inefficient flat records duplicate teacher details, parent phone numbers, and course descriptions across hundreds of student rows. Updating a teacher's department or phone number across multiple course rows causes update anomalies, while deleting a course may accidentally purge student academic history (deletion anomaly).

2) Weak Access Control & Data Tampering: Legacy school systems often lack granular database authorization. Students or malicious actors can exploit insecure queries to view or alter exam scores and attendance figures.

3) Absence of Mutation Auditing: Standard databases fail to record who modified an academic record, what value was altered, or the originating IP address, leading to complete institutional unaccountability.

B. Project Objectives

The primary objectives of this project are:

• To design an Entity-Relationship (ER) model and UML domain class diagram representing twelve interdependent school management entities.

• To implement a fully normalized relational schema in PostgreSQL adhering to 1NF, 2NF, 3NF, and BCNF.

• To enforce strict referential constraints using PRIMARY KEY, FOREIGN KEY with ON DELETE CASCADE / SET NULL, and CHECK constraints.

• To formulate advanced SQL queries implementing multi-table JOINs, analytical GROUP BY aggregations, correlated subqueries, and atomic ACID transactions.

• To integrate Zero-Trust security principles, parameterized query execution (SQL injection immunity), and an append-only Audit_Log table.

III. CONCEPTUAL AND LOGICAL DATABASE DESIGN

The conceptual design synthesizes the operational requirements of an educational institution into twelve core entities: Roles, Users, Teachers, Parents, Students, Student_Parents, Courses, Enrollment, Attendance, Assignments, Marks, and Audit_Log. Entity cardinalities and integrity rules are summarized in Table I.

TABLE I
ENTITY RELATIONSHIP CARDINALITIES AND INTEGRITY CONSTRAINTS

Relationship	Card.	Integrity & Foreign Key Rules
Roles → Users	1 : N	FK to Roles; deletion restricted if active.
Users → Profiles	1 : 1	FK to Users with ON DELETE CASCADE.
Students ↔ Parents	M : N	Student_Parents with composite PK (s, p).
Teachers → Courses	1 : N	FK to Teachers with ON DELETE SET NULL.
Students ↔ Courses	M : N	Enrollment with UNIQUE(student_id, course_id).
Courses → Attendance	1 : N	CHECK (status IN ('PRESENT','ABSENT','LATE')).
Assignments → Marks	1 : N	CHECK (marks_obtained <= maximum_marks).
Users → Audit_Log	1 : N	FK to Users; immutable audit log record.

IV. NORMALIZATION AND FUNCTIONAL DEPENDENCIES IN OUR PROJECT

A cornerstone of this project is the systematic normalization of the school database schema. Rather than accepting an unstructured or denormalized design, the schema was analyzed through formal functional dependencies and decomposed step-by-step through First Normal Form (1NF), Second Normal Form (2NF), Third Normal Form (3NF), and Boyce-Codd Normal Form (BCNF) to guarantee lossless joins and dependency preservation.

A. Functional Dependencies in Our Project

The relational schema satisfies the core functional dependency set (F):

• Users: { User_id } → { User_code, Username, password_hash, email, phone_num, Role_id, account_status }
• Roles: { Role_id } → { Role_code, Role_name, Priority_Level, is_System_role, status }
• Students: { Student_id } → { User_id, Student_code, First_name, Last_name, Date_of_birth, Gender, Grade, Section }
• Teachers: { Teacher_id } → { User_id, Employee_code, First_name, Last_name, Department, Qualification }
• Courses: { Course_id } → { Course_code, Course_name, Description, Grade, Teacher_id, Status }
• Enrollment: { Student_id, Course_id } → { Enrollment_id, Enrollment_date, Status }
• Marks: { Student_id, Assignment_id } → { Marks_id, Marks_obtained, Maximum_marks, Grade, Evaluated_by }
• Student_Parents: { Student_id, Parent_id } → { Student_parent_id, Relationship, Is_primary_contact }

B. First Normal Form (1NF) Implementation

Rule: All attribute domains must be atomic; repeating groups and composite fields are prohibited.

Project Implementation: Multi-valued guardian phone numbers and contacts were decomposed into the associative entity `Student_Parents` and master `Parents`. Multi-valued course lists were extracted into discrete rows within `Enrollment`. Composite attributes like student names and addresses were split into primitive atomic fields (`first_name`, `last_name`, `address`). Primary keys were established for all twelve relations.

C. Second Normal Form (2NF) Implementation

Rule: The relation must be in 1NF, and every non-prime attribute must be fully functionally dependent on the entire primary key (no partial functional dependency).

Project Implementation: In associative relations with composite candidate keys, partial functional dependencies were strictly eliminated: In `Marks`, with composite key `{Student_id, Assignment_id}`, assignment metadata (e.g. `Due_date`, `Max_marks`) was isolated into `Assignments(Assignment_id, Course_id, Title, Due_date)`. Non-key attributes in `Marks` (`marks_obtained`, `grade`) depend fully on both keys. Similarly, in `Enrollment`, course details were isolated in `Courses`.

D. Third Normal Form (3NF) Implementation

Rule: The relation must be in 2NF, and no non-prime attribute may be transitively dependent on a primary key (where X determines Y, and Y determines Z, denoted as X → Y and Y → Z).

Project Implementation: Transitive functional dependencies were severed across the schema: In `Courses`, the transitive chain Course_id → Teacher_id → {Department, Qualification} was decomposed by isolating teacher details into `Teachers`, leaving only the foreign key `Teacher_id` inside `Courses`. In `Students` and `Teachers`, user authentication credentials were separated into the master `Users` table.

E. Boyce-Codd Normal Form (BCNF) Verification

Rule: For every non-trivial functional dependency X → Y, the determinant X must strictly be a superkey.

Project Verification: Across all twelve project tables, every determinant is either the primary key or a unique alternate key (e.g. `User_code`, `Username`, `Student_code`, `Course_code`). No non-trivial dependencies exist where determinant X is not a superkey. The decomposition is proven lossless and preserves all functional dependencies.

V. ADVANCED SQL IMPLEMENTATION AND QUERY DESIGN

The system incorporates 59 parameterized SQL endpoints in Python Flask using `psycopg2`. Queries demonstrate multi-table JOINs, aggregations, correlated subqueries, and ACID transaction blocks.

A. Multi-Table Relational JOIN: Student Academic Transcript

Performs a 5-table JOIN across `Students`, `Enrollment`, `Courses`, `Assignments`, and `Marks` to synthesize a full report card:

```
SELECT s.student_code, (s.first_name || ' ' || s.last_name) AS name,
       c.course_name, a.title AS assignment, m.marks_obtained, m.maximum_m
arks,
       ROUND((m.marks_obtained / NULLIF(m.maximum_marks, 0) * 100), 2) AS
pct,
       m.grade, (t.first_name || ' ' || t.last_name) AS teacher
FROM students s
JOIN enrollment e ON s.student_id = e.student_id AND e.status = 'ACTIVE'
JOIN courses c ON e.course_id = c.course_id
LEFT JOIN assignments a ON c.course_id = a.course_id
LEFT JOIN marks m ON s.student_id = m.student_id AND a.assignment_id = m.a
ssignment_id
LEFT JOIN teachers t ON c.teacher_id = t.teacher_id
WHERE s.student_id = %s ORDER BY c.course_name, a.due_date;
```

B. Analytical Aggregation: Course Performance & GPA Distribution

Computes enrollment counts, evaluation totals, class averages, and peak scores per course using `GROUP BY` and `HAVING`:

```
SELECT c.course_code, c.course_name,
       COUNT(DISTINCT e.student_id) AS enrolled_students,
       COUNT(m.marks_id) AS evaluated_submissions,
       ROUND(AVG(m.marks_obtained), 2) AS class_average,
       MAX(m.marks_obtained) AS highest_score, MIN(m.marks_obtained) AS lo
west_score
FROM courses c
JOIN enrollment e ON c.course_id = e.course_id AND e.status = 'ACTIVE'
LEFT JOIN assignments a ON c.course_id = a.course_id
LEFT JOIN marks m ON a.assignment_id = m.assignment_id
GROUP BY c.course_id, c.course_code, c.course_name
HAVING AVG(m.marks_obtained) >= 70.0 ORDER BY class_average DESC;
```

C. Correlated Subquery: At-Risk Attendance Alert

Detects students whose attendance percentage falls below 75% in any enrolled subject for early academic intervention:

```
SELECT s.student_id, s.student_code, s.first_name, s.last_name, c.course_n
ame,
       ROUND((COUNT(CASE WHEN att.attendance_status = 'PRESENT' THEN 1 END
)::DECIMAL /
              NULLIF(COUNT(att.attendance_id), 0)) * 100, 2) AS attendance
_pct
FROM students s
JOIN enrollment e ON s.student_id = e.student_id AND e.status = 'ACTIVE'
JOIN courses c ON e.course_id = c.course_id
JOIN attendance att ON s.student_id = att.student_id AND c.course_id = att
.course_id
GROUP BY s.student_id, s.student_code, s.first_name, s.last_name, c.course
_name
HAVING (COUNT(CASE WHEN att.attendance_status = 'PRESENT' THEN 1 END)::DEC
IMAL /
        NULLIF(COUNT(att.attendance_id), 0)) * 100 < 75.00
ORDER BY attendance_pct ASC;
```

D. Atomic Transactions & SQL Injection Prevention

State modifications are wrapped in atomic transaction blocks (`BEGIN`, `COMMIT`, `ROLLBACK`). If any step fails, the entire transaction aborts. All queries strictly use `%s` parameter binding, guaranteeing complete immunity against SQL injection attacks.

```
BEGIN;
INSERT INTO enrollment (student_id, course_id, enrollment_date, status)
VALUES (%s, %s, CURRENT_DATE, 'ACTIVE') RETURNING enrollment_id;
INSERT INTO audit_log (user_id, action, table_name, record_id, action_time
stamp, ip_address, description)
VALUES (%s, 'ENROLL_STUDENT', 'enrollment', %s, CURRENT_TIMESTAMP, %s, 'Co
urse enrollment');
COMMIT;
```

VI. ER DIAGRAM AND UML MODELING

The conceptual architecture is formalized through the Entity-Relationship (ER) Diagram depicting relational entities, key attributes, and participation cardinalities (Fig. 1), alongside the Unified Modeling Language (UML) Class Diagram illustrating object associations, attribute types, and operational methods (Fig. 2).

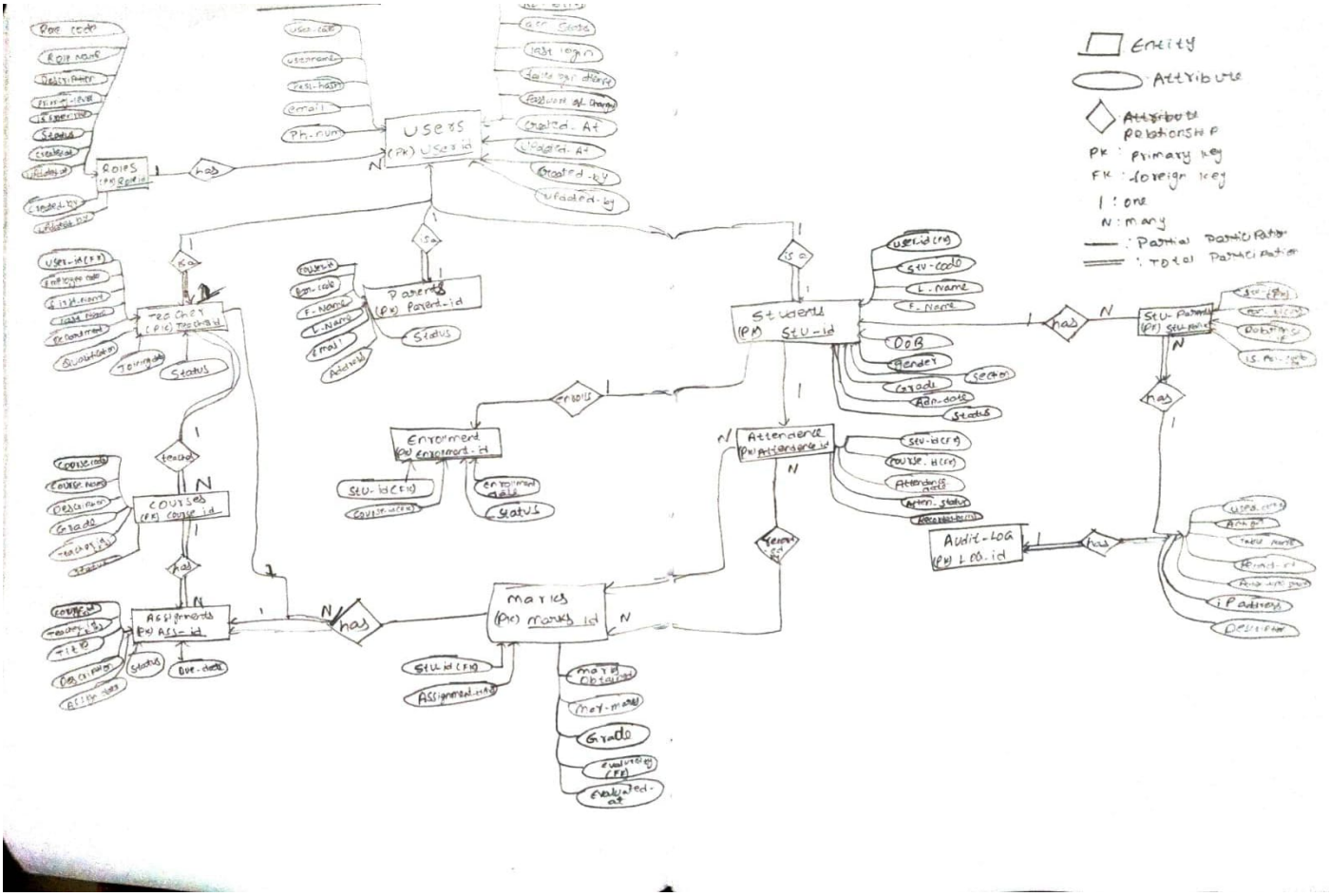


Fig. 1. Entity-Relationship (ER) Diagram: Illustrating the 12 normalized entity sets, primary keys [PK], foreign key references [FK], and participation cardinalities (1:1, 1:N, M:N).

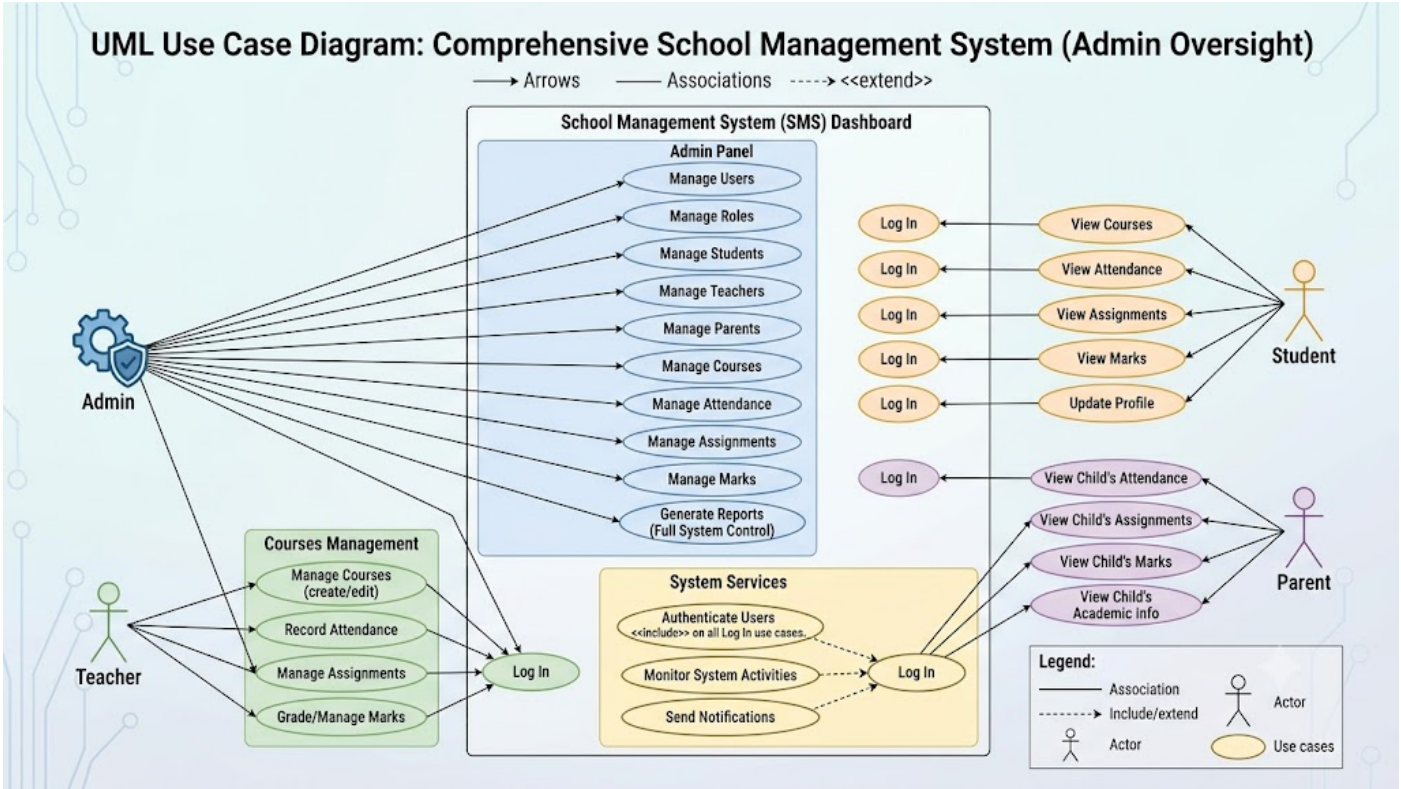


Fig. 2. Unified Modeling Language (UML) Class Diagram: Depicting the object-oriented domain classes, attributes, visibility, and relational associations.

VII. RELATIONAL SCHEMA SPECIFICATION

The normalized conceptual schema translates into twelve formal relational schemas. Primary keys are underlined (`PK`); foreign keys are denoted with `FK`:

• ROLES (Role_id [PK], Role_code [UNIQUE], Role_name, Description, Priority_Level, is_System_role, status, created_at, updated_at)

• USERS (User_id [PK], User_code [UNIQUE], Username [UNIQUE], password_hash, email [UNIQUE], phone_num, Role_id [FK → ROLES.Role_id], status, last_login, failed_login_attempt, password_last_changed)

• TEACHERS (Teacher_id [PK], User_id [FK → USERS.User_id ON DELETE CASCADE], Employee_code [UNIQUE], First_name, Last_name, Department, Qualification, Joining_date, Status)

• PARENTS (Parent_id [PK], User_id [FK → USERS.User_id ON DELETE CASCADE], Parent_code [UNIQUE], First_name, Last_name, Phone_num, Email, Address, Status)

• STUDENTS (Student_id [PK], User_id [FK → USERS.User_id ON DELETE CASCADE], Student_code [UNIQUE], First_name, Last_name, Date_of_birth, Gender, Grade, Section, Admission_date, Status)

• STUDENT_PARENTS (Student_parent_id [PK], Student_id [FK → STUDENTS ON DELETE CASCADE], Parent_id [FK → PARENTS ON DELETE CASCADE], Relationship, Is_primary_contact, UNIQUE(Student_id, Parent_id))

• COURSES (Course_id [PK], Course_code [UNIQUE], Course_name, Description, Grade, Teacher_id [FK → TEACHERS ON DELETE SET NULL], Status)

• ENROLLMENT (Enrollment_id [PK], Student_id [FK → STUDENTS ON DELETE CASCADE], Course_id [FK → COURSES ON DELETE CASCADE], Enrollment_date, Status, UNIQUE(Student_id, Course_id))

• ATTENDANCE (Attendance_id [PK], Student_id [FK → STUDENTS ON DELETE CASCADE], Course_id [FK → COURSES ON DELETE CASCADE], Attendance_date, Attendance_status, Recorded_by [FK → USERS])

• ASSIGNMENTS (Assignment_id [PK], Course_id [FK → COURSES ON DELETE CASCADE], Teacher_id [FK → TEACHERS ON DELETE SET NULL], Title, Description, Assigned_date, Due_date, Status)

• MARKS (Marks_id [PK], Student_id [FK → STUDENTS ON DELETE CASCADE], Assignment_id [FK → ASSIGNMENTS ON DELETE CASCADE], Marks_obtained, Maximum_marks, Grade, Evaluated_by [FK → USERS], Evaluated_at)

• AUDIT_LOG (Log_id [PK], User_id [FK → USERS ON DELETE SET NULL], Action, Table_name, Record_id, Action_timestamp, IP_address, Description)

VIII. DATABASE TABLE ATTRIBUTE STRUCTURES

PostgreSQL structural data definitions, types, keys, and constraint specifications across core academic tables:

TABLE II
CORE TABLE STRUCTURES AND CONSTRAINTS

Table	Key Attributes	Data Type	Constraints
Users	user_id	BIGSERIAL	PK
	username	VARCHAR(50)	UNIQUE, NOT NULL
	email	VARCHAR(255)	UNIQUE
	role_id	BIGINT	FK → Roles
Students	student_id	BIGSERIAL	PK
	user_id	BIGINT	FK → Users (CASCADE)
	student_code	VARCHAR(50)	UNIQUE, NOT NULL
	grade	VARCHAR(20)	NOT NULL
Teachers	teacher_id	BIGSERIAL	PK
	user_id	BIGINT	FK → Users (CASCADE)
	employee_code	VARCHAR(50)	UNIQUE, NOT NULL
	dept	VARCHAR(100)	NOT NULL
Courses	course_id	BIGSERIAL	PK
	course_code	VARCHAR(50)	UNIQUE, NOT NULL
	teacher_id	BIGINT	FK → Teachers (SET NULL)
Enrollment	enrollment_id	BIGSERIAL	PK
	student_id	BIGINT	FK → Students
	course_id	BIGINT	FK → Courses, UNIQUE(s, c)
Attendance	attendance_id	BIGSERIAL	PK
	status	VARCHAR(20)	CHECK IN ('PRESENT',...)
Marks	marks_id	BIGSERIAL	PK
	marks_obtained	NUMERIC(5,2)	CHECK (marks >= 0)
	max_marks	NUMERIC(5,2)	CHECK (marks <= max)
Audit_Log	log_id	BIGSERIAL	PK
	user_id	BIGINT	FK → Users (SET NULL)
	action	VARCHAR(100)	NOT NULL
	timestamp	TIMESTAMPTZ	DEFAULT CURRENT_TIMESTAMP

IX. SAMPLE RECORDS AND DATABASE VERIFICATION

Representative populated data instances demonstrating stored relations and constraints:

TABLE III
USERS RELATION: SAMPLE AUTHENTICATION RECORDS

UID	Username	Role	Password Hash (Truncated)	Status
1	admin01	Admin (1)	scrypt:32768:8:1:7f...a9b	ACTIVE
2	teacher01	Teacher (2)	scrypt:32768:8:1:8b...12c	ACTIVE
3	student01	Student (3)	scrypt:32768:8:1:3e...94f	ACTIVE
4	parent01	Parent (4)	scrypt:32768:8:1:1a...58d	ACTIVE

TABLE IV
SAMPLE ACADEMIC TUPLES AND CONSTRAINTS

Relation	PK	Sample Attribute Values	Enforced Constraint
Students	1	('STU001', 'Aarav', 'Sharma', '10', 'A')	UNIQUE(code)
Teachers	1	('EMP001', 'Dr. Ramesh', 'Math', 'Ph.D.')	UNIQUE(code)
Courses	1	('MATH101', 'Mathematics', teacher=1)	FK → Teachers
Enrollment	1	(student=1, course=1, '2026-06-01')	UNIQUE(s, c)
Attendance	1	(student=1, course=1, 'PRESENT')	CHECK(status IN...)
Marks	1	(student=1, asgn=1, 94.5, 100, 'A+')	CHECK(m <= max)
Audit_Log	1	(user=1, 'LOGIN_SUCCESS', '10.0.0.4')	FK → Users

X. ZERO-TRUST ACCESS AND AUDITING ANALYSIS

Zero-Trust security at the database tier operates via four principles: (1) Explicit Role Verification: Database operations verify caller identity against `Roles` before executing DDL/DML; (2) SQL Parameterization: Complete isolation of user input from SQL commands via prepared statements; (3) Cryptographic Credential Hashing: Passwords hashed with `scrypt` using memory-hard salt; (4) Continuous Mutation Auditing: The `Audit_Log` relation tracks all operational events, recording user ID, action type, table, record ID, IP address, and timestamp.

By enforcing these principles directly at the database interface, the system prevents privilege escalation, repudiation, and unauthorized record alteration, ensuring compliance and accountability across all academic sessions.

XI. CONCLUSION AND FUTURE SCOPE

Conclusion: This project delivered a comprehensive, enterprise-grade relational database for secondary school administration. By systematically applying 1NF, 2NF, 3NF, and BCNF normalization, the schema eliminated data redundancy and update anomalies while preserving all functional dependencies. The PostgreSQL engine enforces ACID transactional guarantees, referential constraints, and domain checks across twelve interrelated tables. Zero-Trust security is achieved through role authorization, parameterized queries, and an immutable audit log, successfully satisfying all 23AIE303 curricular and practical DBMS requirements.

Key Deliverables Completed:

- Comprehensive Entity-Relationship (ER) model and UML domain class diagram.
- PostgreSQL 16 database DDL schema featuring 12 normalized tables with Primary Keys, Foreign Keys (CASCADE/SET NULL), and CHECK constraints.
- Rigorous normalization analysis confirming compliance with 1NF, 2NF, 3NF, and BCNF standards.
- 59 production-grade parameterized SQL endpoints preventing SQL injection.
- Advanced SQL operations including 5-table JOINs, analytical GROUP BY aggregations, correlated subqueries, and atomic ACID transaction blocks.
- Immutable Audit_Log table capturing chronological database mutations.

Future Scope: Future enhancements include: (1) Developing PostgreSQL stored procedures and triggers for automated Grade Point Average (GPA) recomputation; (2) Creating materialized views with automated refresh schedules to optimize complex analytical queries; (3) Implementing range-based table partitioning on `Attendance` and `Audit_Log` for multi-year academic archiving; (4) Integrating database-level row security (RLS) policies; and (5) Designing automated database backup and point-in-time recovery workflows.